```
import os
import re
import time
import logging
import asyncio
import sqlite3
import json
import random
import string
from datetime import datetime, timedelta

from pyrogram import Client, filters
from pyrogram.types import (
    Message, CallbackQuery,
    InlineKeyboardMarkup,
InlineKeyboardButton
)
from pyrogram.errors import FloodWait,
MessageNotModified

# ----------------------------------------
---------------------------------
# Config
# ----------------------------------------
---------------------------------
BOT_TOKEN  =
os.environ["TELEGRAM_BOT_TOKEN"]
API_ID     =
int(os.environ["TELEGRAM_API_ID"])
API_HASH   =
```

```
os.environ["TELEGRAM_API_HASH"]
OWNER_ID   = int(os.environ["OWNER_ID"])

PIC_CHANNEL = "@GrokBotsPics"
PIC_MSG_ID  = 133

logging.basicConfig(format="%(asctime)s - %
(levelname)s - %(message)s",
level=logging.INFO)
logger = logging.getLogger(__name__)

app = Client("protection_bot",
api_id=API_ID, api_hash=API_HASH,
bot_token=BOT_TOKEN)

# ----------------------------------------
---------------------------------
# Database (SQLite)
# ----------------------------------------
---------------------------------
DB_PATH = "bot_data.db"

def db():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def db_init():
    with db() as c:
        c.execute("""CREATE TABLE IF NOT
EXISTS props (
            key TEXT PRIMARY KEY, value
TEXT
        )""")
```

```
        c.execute("""CREATE TABLE IF NOT
EXISTS user_state (
            user_id INTEGER PRIMARY KEY,
state TEXT, temp TEXT
        )""")
        c.execute("""CREATE TABLE IF NOT
EXISTS last_msg (
            user_id INTEGER PRIMARY KEY,
msg_id INTEGER
        )""")
        c.commit()

def get_prop(key, default=None):
    with db() as c:
        row = c.execute("SELECT value FROM
props WHERE key=?", (key,)).fetchone()
        if row is None:
            return default
        try:
            return json.loads(row["value"])
        except Exception:
            return row["value"]

def set_prop(key, value):
    with db() as c:
        c.execute("INSERT OR REPLACE INTO
props(key,value) VALUES(?,?)",
                  (key, json.dumps(value)))
        c.commit()

def get_state(user_id):
    with db() as c:
        row = c.execute("SELECT state,temp
FROM user_state WHERE user_id=?",
```

```
(user_id,)).fetchone()
        if row:
            return row["state"] or "",
row["temp"] or ""
        return "", ""

def set_state(user_id, state, temp=""):
    with db() as c:
        c.execute("INSERT OR REPLACE INTO
user_state(user_id,state,temp)
VALUES(?,?,?)",
                  (user_id, state, temp))
        c.commit()

def get_last_msg(user_id):
    with db() as c:
        row = c.execute("SELECT msg_id FROM
last_msg WHERE user_id=?",
(user_id,)).fetchone()
        return row["msg_id"] if row else
None

def set_last_msg(user_id, msg_id):
    with db() as c:
        c.execute("INSERT OR REPLACE INTO
last_msg(user_id,msg_id) VALUES(?,?)",
(user_id, msg_id))
        c.commit()

# ----------------------------------------
---------------------------------
# Small Caps Font
# ----------------------------------------
---------------------------------
```

```
SMALL_CAPS = {

'a':'ᴀ','b':'ʙ','c':'ᴄ','d':'ᴅ','e':'ᴇ','f'
:'ꜰ','g':'ɢ','h':'ʜ',

'i':'ɪ','j':'ᴊ','k':'ᴋ','l':'ʟ','m':'ᴍ','n'
:'ɴ','o':'ᴏ','p':'ᴘ',

'q':'ǫ','r':'ʀ','s':'s','t':'ᴛ','u':'ᴜ','v'
:'ᴠ','w':'ᴡ','x':'x',
    'y':'ʏ','z':'ᴢ'
}

def apply_font(text: str) -> str:
    if not text:
        return ""
    result = ""
    capitalize_next = True
    in_code = False
    in_tag = False
    tag_buf = ""
    for ch in text:
        if ch == '<' and not in_code:
            tag_buf = "<"; in_tag = True;
continue
        if in_tag:
            tag_buf += ch
            if ch == '>':
                result += tag_buf
                lt = tag_buf.lower()
                if lt == "<code>": in_code
= True
                if lt == "</code>": in_code
= False
```

```
                in_tag = False; tag_buf =
""
            continue
        if in_code:
            result += ch; continue
        if ch.isalpha():
            if capitalize_next:
                result += ch.upper();
capitalize_next = False
            else:
                result +=
SMALL_CAPS.get(ch.lower(), ch.lower())
        else:
            result += ch
            if ch in " \n_.,:;!?-()[]":
                capitalize_next = True
    return result

def safe_name(user) -> str:
    fn = user.first_name or "User"
    ln = (" " + user.last_name) if
user.last_name else ""
    return (fn + ln).strip()

def format_date(ms) -> str:
    if not ms:
        return "Unknown"
    d = datetime.fromtimestamp(ms / 1000)
    return f"{d.day}/{d.month}/{d.year}"

def gen_uid(admin_id: int) -> str:
    suffix = str(admin_id)[-3:]
    rand =
''.join(random.choices(string.ascii_lowerca
```

```
se + string.digits, k=8))
    return f"a{suffix}{rand}"

# ----------------------------------------
--------------------------------
# Admin / Ban helpers
# ----------------------------------------
--------------------------------
def get_admins():
    lst = get_prop("admins_list", [])
    if OWNER_ID not in lst:
        lst.append(OWNER_ID)
    return lst

def is_admin(uid: int) -> bool:
    if uid == OWNER_ID:
        return True
    lst = get_admins()
    if uid not in lst:
        return False
    plan = get_prop(f"admin_plan_{uid}")
    if plan and plan.get("end_time", 0) <
int(time.time() * 1000):
        return False
    return True

def is_banned(uid: int) -> bool:
    return bool(get_prop(f"banned_{uid}",
False))

def set_banned(uid: int, val: bool):
    set_prop(f"banned_{uid}", val)

def check_daily_limit(uid: int) -> bool:
```

```
    if is_admin(uid):
        return True
    limit = get_prop(f"daily_limit_{uid}")
    if limit is None or limit ==
"unlimited" or limit == 0:
        return True
    today = datetime.now().strftime("%Y-%m-
%d")
    if get_prop(f"daily_date_{uid}") !=
today:
        set_prop(f"daily_count_{uid}", 0)
        set_prop(f"daily_date_{uid}",
today)
    count = get_prop(f"daily_count_{uid}",
0)
    if count >= limit:
        return False
    set_prop(f"daily_count_{uid}", count +
1)
    return True

# ----------------------------------------
---------------------------------
# Keyboards
# ----------------------------------------
---------------------------------
def btn(text, cb_or_url, use_url=False):
    t = apply_font(text)
    if use_url or (isinstance(cb_or_url,
str) and (cb_or_url.startswith("http") or
cb_or_url.startswith("tg://"))):
        return InlineKeyboardButton(t,
url=cb_or_url)
    return InlineKeyboardButton(t,
```

```
callback_data=cb_or_url)

def main_kb(uid):
    rows = []
    if uid == OWNER_ID:
        rows.append([btn("👑 Owner Panel",
"owner_main")])
    elif is_admin(uid):
        rows.append([btn("🛡️ Admin Panel",
"admin_main")])
    return InlineKeyboardMarkup(rows) if
rows else None

def owner_kb():
    maint = get_prop("maintenance_mode",
False)
    return InlineKeyboardMarkup([
        [btn("👨‍💼 Manage Admins",
"own_admins_menu"), btn("📊 Global
Statistics", "own_stats")],
        [btn("👥 Manage Users",
"adm_manage_user_0"), btn("📣 Broadcast",
"own_bc")],
        [btn(f"{'🔴 Maintenance: ON' if
maint else '🟢 Maintenance: OFF'}",
"own_toggle_maint")],
        [btn("📝 Set Log Channel",
"own_set_log"), btn("📚 Help",
"help_menu")],
        [btn("🏠 Back To Dashboard",
"go_home")],
    ])

def admin_kb(uid):
```

```
    notify = get_prop(f"user_notify_{uid}",
False)
    return InlineKeyboardMarkup([
        [btn("⚙️ Channel Configurations",
"config_menu")],
        [btn("📊 My Statistics",
"adm_my_stats"), btn("👥 Manage Users",
"adm_manage_user_0")],
        [btn(f"{'🔔 Alerts: ON' if notify
else '🔕 Alerts: OFF'}",
"adm_toggle_notify")],
        [btn("📚 Help", "help_menu")],
        [btn("🏠 Back To Dashboard",
"go_home")],
    ])

def user_manage_kb(uid):
    return InlineKeyboardMarkup([
        [btn(f"🚫 Ban {uid}",
f"act_ban_{uid}"), btn(f"✅ Unban {uid}",
f"act_unban_{uid}")],
        [btn("📈 Set Daily Limit",
f"act_limit_{uid}"), btn("💬 Private
Message", f"act_pm_{uid}")],
        [btn("🔙 Back", "go_home")],
    ])

def back_kb(uid):
    return InlineKeyboardMarkup([[btn("🔙
Return Back", "owner_main" if uid ==
OWNER_ID else "admin_main")]])

# ----------------------------------------
---------------------------------
```

```
# send_msg helper (edit if callback, else
send with pic)
# ----------------------------------------
---------------------------------
async def send_msg(client, chat_id, text,
markup=None, is_cb=False, cb_msg=None):
    final = apply_font(text)
    last = get_last_msg(chat_id)

    if is_cb and cb_msg:
        mid = cb_msg.id
        try:
            if cb_msg.photo or cb_msg.video
or cb_msg.document or cb_msg.animation:
                await
cb_msg.edit_caption(final,
parse_mode="HTML", reply_markup=markup)
            else:
                await
cb_msg.edit_text(final, parse_mode="HTML",
reply_markup=markup,

disable_web_page_preview=True)
            return mid
        except MessageNotModified:
            return mid
        except Exception:
            pass

    if last:
        try:
            await
client.delete_messages(chat_id, last)
        except Exception:
```

```
            pass

    try:
        res = await client.copy_message(
            chat_id,
from_chat_id=PIC_CHANNEL,
message_id=PIC_MSG_ID,
            caption=final,
parse_mode="HTML", reply_markup=markup
        )
        set_last_msg(chat_id, res.id)
        return res.id
    except Exception:
        res = await
client.send_message(chat_id, final,
parse_mode="HTML",

reply_markup=markup,

disable_web_page_preview=True)
        set_last_msg(chat_id, res.id)
        return res.id

# ----------------------------------------
---------------------------------
# Config Menu
# ----------------------------------------
---------------------------------
async def show_config_menu(client, chat_id,
uid, is_cb=False, cb_msg=None):
    pairs = get_prop(f"channels_{uid}", [])
    limit = 999 if uid == OWNER_ID else
get_prop(f"chan_limit_{uid}", 0)
```

```
    txt = ("<b>⚙️ Channel
Configurations</b>\n\n"
           "<blockquote><b>Link your source
and target channels below to automate "
           "forwarding.</b>
</blockquote>\n\n"
           f"<b>📈 Pairs Limit:</b> <code>
{len(pairs)} / "
           f"{'Unlimited' if limit == 999
else limit}</code>\n\n")

    rows = []
    if not pairs:
        txt += "<i>No channels connected
yet.</i>\n"
    else:
        for i, p in enumerate(pairs):
            txt += f"<b>{i+1}. Config:
</b>\n🎯 Target: <code>{p['target']}
</code>\n📣 Source: <code>{p['source']}
</code>\n\n"
            rows.append([btn(f"❌ Remove
Config {i+1}", f"rem_pair_{i}")])

    if len(pairs) < limit:
        rows.append([btn("➕ Link New
Channels", "add_pair_start")])
    elif uid != OWNER_ID:
        txt += "\n⚠️ <i>Channel limit
reached. Contact Owner for upgrade.</i>"

    rows.append([btn("🔙 Back To Panel",
"owner_main" if uid == OWNER_ID else
"admin_main")])
```

```
    await send_msg(client, chat_id, txt,
InlineKeyboardMarkup(rows), is_cb, cb_msg)

# ----------------------------------------
--------------------------------
# /start handler
# ----------------------------------------
--------------------------------
@app.on_message(filters.command("start") &
filters.private)
async def start_handler(client: Client,
message: Message):
    uid = message.from_user.id
    text = message.text or ""

    # Track user join
    if not get_prop(f"join_date_{uid}"):
        set_prop(f"join_date_{uid}",
int(time.time() * 1000))
    if not get_prop(f"uname_{uid}"):
        set_prop(f"uname_{uid}",
safe_name(message.from_user))

    # Deep link media: /start vid_XXXX
    if "vid_" in text:
        unique_id = text.split("vid_")
[1].strip()
        media_data =
get_prop(f"m_{unique_id}")

        if not media_data:
            await send_msg(client, uid,
                "<b>❌ Deprecated Data
Link</b>\n\n"
```

```
                "<blockquote><b>This
content is no longer available.</b>
</blockquote>")
            return

        if not check_daily_limit(uid):
            await send_msg(client, uid,
                "<b>❌ Daily Limit
Reached</b>\n\n"
                "<blockquote><b>You have
exhausted your daily media quota. "
                "Contact admin.</b>
</blockquote>")
            return

        admin_uid =
media_data.get("admin_uid")
        if admin_uid:
            if not
get_prop(f"joined_via_{uid}"):

set_prop(f"joined_via_{uid}", admin_uid)
            views =
get_prop(f"views_own_{admin_uid}", 0) + 1

set_prop(f"views_own_{admin_uid}", views)
            viewers =
get_prop(f"viewers_{admin_uid}", [])
            if uid not in viewers:
                viewers.append(uid)

set_prop(f"viewers_{admin_uid}", viewers)

            # Notify admin first time
```

```
            if not
get_prop(f"notified_first_{uid}"):

set_prop(f"notified_first_{uid}", True)
                if
get_prop(f"user_notify_{admin_uid}"):
                    kb =
InlineKeyboardMarkup([[btn("⚙️ Manage
User", f"adm_uid_{uid}")]])
                    await
client.send_message(
                        admin_uid,
                        apply_font(f"<b>🔔
New User Alert</b>\n\n"
                                   f"<b>👤
Name:</b> <code>
{safe_name(message.from_user)}</code>\n"
                                   f"<b>🆔
ID:</b> <code>{uid}</code>"),
                        parse_mode="HTML",
reply_markup=kb
                    )

        set_prop(f"views_by_{uid}",
get_prop(f"views_by_{uid}", 0) + 1)

        # ASCII loading animation
        frames = [
            "____ \n| __ \n|__] ",
            "____ ____ \n| __ |  | \n|__]
|__| ",
            "____ ____ ___  \n| __ |  | |
\\ \n|__] |__| |__/ ",
            "____ ____ ___  ____ \n| __ |
```

```
| |  \\ [__  \n|__] |__| |__/ ___] ",
            "____ ____ ___  ____ ____ \n|
__ |  | |  \\ [__  |  | \n|__] |__| |__/
___] |__| ",
            "____ ____ ___  ____ ____ _  _
\n| __ |  | |  \\ [__  |  | |\\ | \n|__]
|__| |__/ ___] |__| | \\| ",
        ]
        mid = await send_msg(client, uid,
f"<code>{frames[0]}</code>")
        for frame in frames[1:]:
            await asyncio.sleep(1.8)
            try:
                await
client.edit_message_caption(uid, mid,
caption=f"<code>{frame}</code>",
parse_mode="HTML")
            except Exception:
                try:
                    await
client.edit_message_text(uid, mid, text=f"
<code>{frame}</code>", parse_mode="HTML")
                except Exception:
                    pass
        try:
            await
client.delete_messages(uid, mid)
        except Exception:
            pass

        # Send protected media
        mtype = media_data.get("type",
"video")
        fid = media_data.get("file_id")
```

```
        cap =
apply_font(media_data.get("caption", ""))
or None
        opts = dict(chat_id=uid,
parse_mode="HTML", protect_content=True)
        if cap:
            opts["caption"] = cap
        try:
            if mtype == "photo":
                await
client.send_photo(**opts, photo=fid)
            elif mtype == "document":
                await
client.send_document(**opts, document=fid)
            else:
                await
client.send_video(**opts, video=fid)
        except Exception as e:
            logger.exception("Media send
failed")
        return

    # Maintenance check
    if get_prop("maintenance_mode", False)
and not is_admin(uid):
        await send_msg(client, uid,
            "<b>🛠️ System Under
Maintenance</b>\n\n"
            "<blockquote><b>Please check
back shortly.</b></blockquote>")
        return

    # Ban check
    if is_banned(uid):
```

```
        await send_msg(client, uid,
            "<b>🚫 Security Alert</b>\n\n"
            "<blockquote><b>Your access has
been permanently restricted.</b>
</blockquote>")
        return

    # Unauthorized user
    if not is_admin(uid):
        kb =
InlineKeyboardMarkup([[btn(f"📞 Contact
Owner To Buy", f"tg://user?id={OWNER_ID}",
True)]])
        await send_msg(client, uid,
            f"<b>✨ Premium Bot
Network</b>\n\n"
            f"<blockquote><b>👋 Hello
{safe_name(message.from_user)}!\n\n"
            f"This is an elite private
auto-forwarding and content protection
system.\n\n"
            f"You are currently not
authorized. Purchase an Administrator
Subscription "
            f"to get access.</b>
</blockquote>", kb)
        return

    set_state(uid, "")
    role = "Master Owner" if uid ==
OWNER_ID else "Administrator"
    await send_msg(client, uid,
        f"<b>✨ Welcome To The Premium
Dashboard</b>\n\n"
```

```
        f"<b>👤 Name:</b> <code>
{safe_name(message.from_user)}</code>\n"
        f"<b>🆔 System ID:</b> <code>{uid}
</code>\n"
        f"<b>🛡️ Role:</b> <code>{role}
</code>\n\n"
        f"<blockquote><b>Use the panel
below to access system controls.</b>
</blockquote>",
        main_kb(uid))

# ----------------------------------------
---------------------------------
# Callback Query Handler
# ----------------------------------------
---------------------------------
@app.on_callback_query()
async def cb_handler(client: Client, query:
CallbackQuery):
    uid = query.from_user.id
    d = query.data
    msg = query.message

    # Maintenance / ban gate
    if get_prop("maintenance_mode", False)
and not is_admin(uid):
        await query.answer("System under
maintenance. Back shortly.",
show_alert=True)
        return
    if is_banned(uid):
        await query.answer("Access Denied.
Account restricted.", show_alert=True)
        return
```

```
    if not is_admin(uid):
        await query.answer("Unauthorized.
Purchase a subscription.", show_alert=True)
        return

    await query.answer()

    # vid_ callback → redirect to deep
link
    if d.startswith("vid_"):
        unique_id = d.split("_", 1)[1]
        me = await client.get_me()
        url = f"https://t.me/{me.username}?
start=vid_{unique_id}"
        await query.answer(url=url)
        return

    if d == "go_home":
        set_state(uid, "")
        role = "Master Owner" if uid ==
OWNER_ID else "Administrator"
        await send_msg(client, uid,
            f"<b>✨ Welcome To The Premium
Dashboard</b>\n\n"
            f"<b>👤 Name:</b> <code>
{safe_name(query.from_user)}</code>\n"
            f"<b>🆔 System ID:</b> <code>
{uid}</code>\n"
            f"<b>🛡️ Role:</b> <code>{role}
</code>\n\n"
            f"<blockquote><b>Use the panel
below to access system controls.</b>
</blockquote>",
            main_kb(uid), True, msg)
```

```
        return

    if d == "help_menu":
        help_txt = ("<blockquote><b>📚
System Operations Tutorial</b>\n\n"
                    "<b>1. Channel
Configurations:</b>\n"
                    "• Link New Channels →
send Target ID (-100...) → send Source
ID.\n"
                    "• Target = where you
upload raw media.\n"
                    "• Source = where
protected links get posted.\n\n"
                    "<b>2. Managing Users:
</b>\n"
                    "• Ban/Unban users, set
daily view limits, send private
messages.\n"
                    "• Send 0 for Unlimited
daily limit.\n\n"
                    "<b>3. Owner
Exclusives:</b>\n"
                    "• Add/Remove admins
with expiry duration (e.g. 30d).\n"
                    "• Set Log Channel for
backup archives.\n"
                    "• Broadcast to all
users.</blockquote>")
        await send_msg(client, uid,
help_txt, back_kb(uid), True, msg)
        return

    if d == "config_menu":
```

```
        await show_config_menu(client, uid,
uid, True, msg)
        return

    if d == "add_pair_start":
        set_state(uid, "wait_target")
        await send_msg(client, uid,
            "<b>🎯 Linking New Target
Channel</b>\n\n"
            "<blockquote><b>Send the Target
Channel ID (must start with -100).</b>
</blockquote>",
            InlineKeyboardMarkup([[btn("❌
Cancel", "config_menu")]]), True, msg)
        return

    if d.startswith("rem_pair_"):
        idx = int(d.split("_")[2])
        pairs = get_prop(f"channels_{uid}",
[])
        if 0 <= idx < len(pairs):
            pairs.pop(idx)
            set_prop(f"channels_{uid}",
pairs)
        await show_config_menu(client, uid,
uid, True, msg)
        return

    if d == "owner_main" and uid ==
OWNER_ID:
        set_state(uid, "")
        await send_msg(client, uid, "<b>👑
Master Owner Dashboard</b>", owner_kb(),
True, msg)
```

```
        return

    if d == "admin_main" and is_admin(uid):
        set_state(uid, "")
        await send_msg(client, uid, "<b>🛡️
Administration Dashboard</b>",
admin_kb(uid), True, msg)
        return

    if d == "own_toggle_maint" and uid ==
OWNER_ID:
        set_prop("maintenance_mode", not
get_prop("maintenance_mode", False))
        await send_msg(client, uid, "<b>👑
Master Owner Dashboard</b>", owner_kb(),
True, msg)
        return

    if d == "adm_toggle_notify":
        set_prop(f"user_notify_{uid}", not
get_prop(f"user_notify_{uid}", False))
        await send_msg(client, uid, "<b>🛡️
Administration Dashboard</b>",
admin_kb(uid), True, msg)
        return

    if d == "own_admins_menu" and uid ==
OWNER_ID:
        kb = InlineKeyboardMarkup([
            [btn("➕ Create Admin",
"own_admin_add"), btn("🗑️ Revoke Admin",
"own_admin_del")],
            [btn("📋 View Admins",
"own_admin_list")],
```

```
            [btn("🔙 Return",
"owner_main")],
        ])
        await send_msg(client, uid,
            "<b>👨‍💼 Administrator Control
Protocol</b>\n\n"
            "<blockquote><b>Select an
operation below.</b></blockquote>", kb,
True, msg)
        return

    if d == "own_admin_add" and uid ==
OWNER_ID:
        set_state(uid, "wait_admin_add")
        await send_msg(client, uid,
            "<b>➕ Add Admin (Step 1/2)
</b>\n\n"
            "<blockquote><b>Enter the User
ID to elevate to Administrator:</b>
</blockquote>",
            InlineKeyboardMarkup([[btn("❌
Cancel", "own_admins_menu")]]), True, msg)
        return

    if d == "own_admin_del" and uid ==
OWNER_ID:
        set_state(uid, "wait_admin_del")
        await send_msg(client, uid,
            "<b>🗑️ Revoke Admin</b>\n\n"
            "<blockquote><b>Enter the User
ID to remove from Administrators:</b>
</blockquote>",
            InlineKeyboardMarkup([[btn("❌
Cancel", "own_admins_menu")]]), True, msg)
```

```
        return

    if d == "own_admin_list" and uid ==
OWNER_ID:
        admins = get_admins()
        txt = f"<b>📋 Active
Administrators</b>\n\n<b>👑 Owner:</b>
<code>{OWNER_ID}</code>\n"
        for a in admins:
            if a == OWNER_ID:
                continue
            plan =
get_prop(f"admin_plan_{a}")
            if plan and
plan.get("end_time", 0) > int(time.time() *
1000):
                left =
int((plan["end_time"] - time.time() * 1000)
/ 86400000) + 1
                status = f"{left} Days
Remaining"
            else:
                status = "Expired"
            txt += f"<b>🛡️ ID:</b> <code>
{a}</code> ({status})\n"
        txt += "\n<blockquote><b>Send an
Admin ID below to view their analytics.</b>
</blockquote>"
        set_state(uid, "wait_manage_admin")
        await send_msg(client, uid, txt,
            InlineKeyboardMarkup([[btn("🔙
Return", "own_admins_menu")]]), True, msg)
        return
```

```
    if d.startswith("adm_manage_user_"):
        page = int(d.split("_")[3]) or 0
        if uid == OWNER_ID:
            # All users in DB
            with db() as c:
                rows = c.execute("SELECT
DISTINCT user_id FROM
user_state").fetchall()
            users_list = [r["user_id"] for
r in rows]
        else:
            users_list =
get_prop(f"viewers_{uid}", [])

        total = len(users_list)
        start = page * 25
        sliced = users_list[start:start+25]

        txt = f"<b>👥 User Directory (Page
{page+1})</b>\n\n<blockquote><b>Copy an ID
and send it below.</b></blockquote>\n\n"
        if not sliced:
            txt += "<i>No users found.
</i>\n"
        for u in sliced:
            name = get_prop(f"uname_{u}",
"User")
            txt += f"👤 {name} - <code>{u}
</code>\n"

        nav = []
        if page > 0:
            nav.append(btn("◀️ Prev",
f"adm_manage_user_{page-1}"))
```

```
        if start + 25 < total:
            nav.append(btn("Next ▶️",
f"adm_manage_user_{page+1}"))

        rows_kb = []
        if nav:
            rows_kb.append(nav)
        rows_kb.append([btn("🔙 Return",
"owner_main" if uid == OWNER_ID else
"admin_main")])
        set_state(uid, "wait_manage_uid")
        await send_msg(client, uid, txt,
InlineKeyboardMarkup(rows_kb), True, msg)
        return

    if d.startswith("adm_uid_"):
        target = int(d.split("_")[2])
        await _send_user_stats(client, uid,
target, True, msg)
        return

    if d.startswith("act_ban_"):
        target = int(d.split("_")[2])
        set_banned(target, True)
        await send_msg(client, uid, "<b>✅
User has been banned.</b>",
user_manage_kb(target), True, msg)
        return

    if d.startswith("act_unban_"):
        target = int(d.split("_")[2])
        set_banned(target, False)
        await send_msg(client, uid, "<b>✅
Ban has been lifted.</b>",
```

```
user_manage_kb(target), True, msg)
        return

    if d.startswith("act_limit_"):
        target = d.split("_")[2]
        set_state(uid,
f"wait_limit_{target}")
        await send_msg(client, uid,
            f"<b>⚙️ Set Daily Limit for
<code>{target}</code></b>\n\n"
            "<blockquote><b>Enter daily
quota (0 = Unlimited):</b></blockquote>",
            InlineKeyboardMarkup([[btn("❌
Cancel", f"adm_uid_{target}")]]), True,
msg)
        return

    if d.startswith("act_pm_"):
        target = d.split("_")[2]
        set_state(uid, f"wait_pm_{target}")
        await send_msg(client, uid,
            f"<b>💬 Private Message to
<code>{target}</code></b>\n\n"
            "<blockquote><b>Type your
message now:</b></blockquote>",
            user_manage_kb(int(target)),
True, msg)
        return

    if d == "adm_my_stats":
        pairs = get_prop(f"channels_{uid}",
[])
        viewers =
get_prop(f"viewers_{uid}", [])
```

```
        joined =
format_date(get_prop(f"join_date_{uid}"))
        txt = (f"<b>📊 My
Statistics</b>\n\n"
               f"<b>📅 Since:</b> <code>
{joined}</code>\n"
               f"<b>📡 Active Pairs:</b>
<code>{len(pairs)}</code>\n"
               f"<b>🔄 Forwarded:</b>
<code>{get_prop(f'fwd_{uid}', 0)}
Files</code>\n"
               f"<b>▶️ Views:</b> <code>
{get_prop(f'views_own_{uid}', 0)}</code>\n"
               f"<b>👥 Total Users:</b>
<code>{len(viewers)}</code>")
        await send_msg(client, uid, txt,
back_kb(uid), True, msg)
        return

    if d == "own_stats" and uid ==
OWNER_ID:
        log_ch = get_prop("admin_log_chan",
"Unassigned")
        txt = f"<b>📊 Global
Analytics</b>\n\n<b>📝 Log Channel:</b>
<code>{log_ch}</code>\n\n"
        for i, aid in
enumerate(get_admins()):
            pairs =
get_prop(f"channels_{aid}", [])
            if pairs and is_admin(aid):
                viewers =
get_prop(f"viewers_{aid}", [])
                txt += (f"<b>{i+1}. ID:</b>
```

```
<code>{aid}</code>\n"
                        f"<b>📡 Pairs:</b>
<code>{len(pairs)}</code> | "
                        f"<b>🔄 Fwd:</b>
<code>{get_prop(f'fwd_{aid}', 0)}</code> |
"
                        f"<b>👥 Users:</b>
<code>{len(viewers)}</code>\n\n")
        await send_msg(client, uid, txt,
            InlineKeyboardMarkup([[btn("🔙
Return", "owner_main")]]), True, msg)
        return

    if d == "own_bc" and uid == OWNER_ID:
        set_state(uid, "wait_bc")
        await send_msg(client, uid,
            "<b>📣
Broadcast</b>\n\n<blockquote><b>Send the
message to broadcast to all users:</b>
</blockquote>",
            InlineKeyboardMarkup([[btn("❌
Cancel", "owner_main")]]), True, msg)
        return

    if d == "own_set_log" and uid ==
OWNER_ID:
        set_state(uid, "wait_log_chan")
        await send_msg(client, uid,
            "<b>📝 Set Log
Channel</b>\n\n<blockquote><b>Send the
Channel ID (starting with -100):</b>
</blockquote>",
            InlineKeyboardMarkup([[btn("❌
Cancel", "owner_main")]]), True, msg)
```

```
        return

async def _send_user_stats(client,
from_uid, target_uid, is_cb=False,
cb_msg=None):
    joined =
format_date(get_prop(f"join_date_{target_ui
d}"))
    via =
get_prop(f"joined_via_{target_uid}",
"Direct")
    views =
get_prop(f"views_by_{target_uid}", 0)
    limit =
get_prop(f"daily_limit_{target_uid}")
    limit_txt = "Unlimited" if (limit is
None or limit == "unlimited" or limit == 0)
else str(limit)
    count =
get_prop(f"daily_count_{target_uid}", 0)
    today = datetime.now().strftime("%Y-%m-
%d")
    if get_prop(f"daily_date_{target_uid}")
!= today:
        count = 0

    txt = (f"<b>👤 User Analytics</b>\n\n"
           f"<b>🆔 ID:</b> <code>
{target_uid}</code>\n"
           f"<b>📅 Joined:</b> <code>
{joined}</code>\n"
           f"<b>🔗 Via Admin:</b> <code>
{via}</code>\n"
           f"<b>▶️ Views:</b> <code>{views}
```

```
</code>\n"
           f"<b>📈 Daily Quota:</b> <code>
{count} / {limit_txt}</code>\n"
           f"<b>🚫 Status:</b> <code>
{'Banned' if is_banned(target_uid) else
'Active'}</code>\n\n"
           f"<blockquote><b>Choose an
action below:</b></blockquote>")
    await send_msg(client, from_uid, txt,
user_manage_kb(target_uid), is_cb, cb_msg)

# ----------------------------------------
---------------------------------
# Message Handler (state machine)
# ----------------------------------------
---------------------------------
@app.on_message(filters.private &
~filters.command(["start"]))
async def msg_handler(client: Client,
message: Message):
    uid = message.from_user.id
    text = (message.text or "").strip()
    st, temp = get_state(uid)

    if not is_admin(uid):
        return

    async def cleanup():
        try:
            await message.delete()
        except Exception:
            pass

    if st == "wait_target":
```

```
        await cleanup()
        set_state(uid, "wait_source", text)
        await send_msg(client, uid,
            f"<b>📣 Now Send Source Channel
ID</b>\n\n"
            f"<blockquote><b>Binding to
Target: {text}</b></blockquote>",
            InlineKeyboardMarkup([[btn("❌
Abort", "config_menu")]]))
        return

    if st == "wait_source":
        await cleanup()
        pairs = get_prop(f"channels_{uid}",
[])
        pairs.append({"target":
temp.strip(), "source": text.strip()})
        set_prop(f"channels_{uid}", pairs)
        set_state(uid, "")
        await send_msg(client, uid, "<b>✅
Channels Successfully Linked!</b>",
            InlineKeyboardMarkup([[btn("⚙️
Back to Config", "config_menu")]]))
        return

    if st == "wait_admin_add" and uid ==
OWNER_ID:
        await cleanup()
        try:
            new_ad = int(text)
        except ValueError:
            set_state(uid, "")
            await send_msg(client, uid, "
<b>⚠️ Invalid ID.</b>",
```

```
InlineKeyboardMarkup([[btn("❌ Cancel",
"own_admins_menu")]]))
            return
        admins = get_admins()
        if new_ad in admins and
is_admin(new_ad):
            set_state(uid, "")
            await send_msg(client, uid, "
<b>⚠️ Already an Admin.</b>",

InlineKeyboardMarkup([[btn("❌ Cancel",
"own_admins_menu")]]))
            return
        set_state(uid,
f"wait_admin_duration_{new_ad}",
str(new_ad))
        await send_msg(client, uid,
            "<b>⏳ Set Duration (Step 2/2)
</b>\n\n"
            "<blockquote><b>Enter duration
e.g. 30d, 365d:</b></blockquote>",
            InlineKeyboardMarkup([[btn("❌
Abort", "own_admins_menu")]]))
        return

    if
st.startswith("wait_admin_duration_") and
uid == OWNER_ID:
        await cleanup()
        new_ad = int(st.split("_")[3])
        days_str =
text.lower().replace("d", "").strip()
        try:
```

```
            days = int(days_str)
            assert days > 0
        except Exception:
            set_state(uid, "")
            await send_msg(client, uid, "
<b>⚠️ Invalid format. Use e.g. 30d</b>",

InlineKeyboardMarkup([[btn("🔙 Return",
"own_admins_menu")]]))
            return
        end_time = int((time.time() + days
* 86400) * 1000)
        set_prop(f"admin_plan_{new_ad}",
{"end_time": end_time, "days": days})
        admins = get_admins()
        if new_ad not in admins:
            admins.append(new_ad)
            set_prop("admins_list", admins)
        set_state(uid, "")
        await send_msg(client, uid,
            f"<b>✅ Admin Added for {days}
Days!</b>",
            InlineKeyboardMarkup([[btn("✔️
Done", "own_admins_menu")]]))
        return

    if st == "wait_admin_del" and uid ==
OWNER_ID:
        await cleanup()
        try:
            del_ad = int(text)
        except ValueError:
            set_state(uid, "")
            await send_msg(client, uid, "
```

```
<b>⚠️ Invalid ID.</b>",

InlineKeyboardMarkup([[btn("🔙 Return",
"own_admins_menu")]]))
            return
        if del_ad == OWNER_ID:
            await send_msg(client, uid, "
<b>⚠️ Cannot remove Owner.</b>",

InlineKeyboardMarkup([[btn("🔙 Return",
"own_admins_menu")]]))
            return
        admins = [a for a in get_admins()
if a != del_ad]
        set_prop("admins_list", admins)
        set_prop(f"admin_plan_{del_ad}",
{"end_time": 0})
        set_state(uid, "")
        await send_msg(client, uid, "<b>✅
Admin Revoked.</b>",
            InlineKeyboardMarkup([[btn("✔️
Done", "own_admins_menu")]]))
        return

    if st == "wait_manage_admin" and uid ==
OWNER_ID:
        await cleanup()
        try:
            aid = int(text)
        except ValueError:
            set_state(uid, "")
            await send_msg(client, uid, "
<b>⚠️ Invalid ID.</b>",
```

```
InlineKeyboardMarkup([[btn("🔙 Return",
"own_admins_menu")]]))
            return
        set_state(uid,
f"wait_admin_limit_{aid}")
        pairs = get_prop(f"channels_{aid}",
[])
        viewers =
get_prop(f"viewers_{aid}", [])
        ch_limit =
get_prop(f"chan_limit_{aid}", 0)
        txt = (f"<b>📊 Admin
Details</b>\n\n"
               f"<b>🆔 ID:</b> <code>{aid}
</code>\n"
               f"<b>📡 Pairs:</b> <code>
{len(pairs)}/{ch_limit}</code>\n"
               f"<b>🔄 Forwarded:</b>
<code>{get_prop(f'fwd_{aid}', 0)}</code>\n"
               f"<b>👥 Users:</b> <code>
{len(viewers)}</code>\n\n"
               f"<blockquote><b>Send a
number to set their max channel pairs
limit:</b></blockquote>")
        await send_msg(client, uid, txt,
            InlineKeyboardMarkup([[btn("❌
Abort", "own_admins_menu")]]))
        return

    if st.startswith("wait_admin_limit_")
and uid == OWNER_ID:
        await cleanup()
        aid = st.split("_")[3]
        try:
```

```
            lim = int(text)
            assert lim >= 0
        except Exception:
            set_state(uid, "")
            await send_msg(client, uid, "
<b>⚠️ Invalid number.</b>",

InlineKeyboardMarkup([[btn("🔙 Return",
"own_admins_menu")]]))
            return
        set_prop(f"chan_limit_{aid}", lim)
        set_state(uid, "")
        await send_msg(client, uid,
            f"<b>✅ Channel limit set to
{lim} for admin <code>{aid}</code>.</b>",
            InlineKeyboardMarkup([[btn("✔️
Done", "own_admins_menu")]]))
        return

    if st == "wait_log_chan" and uid ==
OWNER_ID:
        await cleanup()
        set_prop("admin_log_chan", text)
        set_state(uid, "")
        await send_msg(client, uid,
            f"<b>✅ Log Channel Set: <code>
{text}</code></b>",
            InlineKeyboardMarkup([[btn("🔙
Back", "owner_main")]]))
        return

    if st.startswith("wait_limit_"):
        await cleanup()
        target_id = st.split("_")[2]
```

```
        try:
            limit = int(text)
            assert limit >= 0
        except Exception:
            await send_msg(client, uid, "
<b>⚠️ Enter a valid number.</b>",
user_manage_kb(int(target_id)))
            return
        if limit == 0:

set_prop(f"daily_limit_{target_id}",
"unlimited")
            await send_msg(client, uid,
                f"<b>✅ Limit set to
Unlimited for <code>{target_id}</code>.
</b>",

user_manage_kb(int(target_id)))
        else:

set_prop(f"daily_limit_{target_id}", limit)
            await send_msg(client, uid,
                f"<b>✅ Daily limit set to
{limit} for <code>{target_id}</code>.</b>",

user_manage_kb(int(target_id)))
        set_state(uid, "")
        return

    if st == "wait_manage_uid":
        await cleanup()
        try:
            target_id = int(text)
        except ValueError:
```

```
            await send_msg(client, uid, "
<b>⚠️ Invalid ID.</b>", back_kb(uid))
            return
        set_state(uid, "")
        await _send_user_stats(client, uid,
target_id)
        return

    if st.startswith("wait_pm_"):
        target_id = st.split("_")[2]
        try:
            await
client.copy_message(int(target_id), uid,
message.id)
            await cleanup()
            await send_msg(client, uid, "
<b>✅ Message Delivered.</b>",
user_manage_kb(int(target_id)))
        except Exception:
            await send_msg(client, uid, "
<b>❌ Could not deliver — user may have
blocked the bot.</b>",

user_manage_kb(int(target_id)))
        set_state(uid, "")
        return

    if st == "wait_bc" and uid == OWNER_ID:
        set_state(uid, "")
        bc_msg_id = message.id
        mid = await send_msg(client, uid, "
<b>⏳ Broadcasting...</b>")
        with db() as c:
            rows = c.execute("SELECT
```

```
DISTINCT user_id FROM
user_state").fetchall()
        users_all = [r["user_id"] for r in
rows]
        sent = failed = 0
        for u in users_all:
            try:
                await
client.copy_message(u, uid, bc_msg_id)
                sent += 1
            except Exception:
                failed += 1
            await asyncio.sleep(0.035)
        await cleanup()
        try:
            await
client.edit_message_text(uid, mid,
                apply_font(f"<b>📣
Broadcast Done</b>\n\n"
                           f"<b>✅ Sent:
</b> <code>{sent}</code>\n"
                           f"<b>❌ Failed:
</b> <code>{failed}</code>"),
                parse_mode="HTML",

reply_markup=InlineKeyboardMarkup([[btn("✔️
Done", "owner_main")]]))
        except Exception:
            pass
        return

# ----------------------------------------
---------------------------------
# Channel Post Handler (the core protection
```

```
engine)
# ----------------------------------------
--------------------------------
@app.on_message(filters.channel)
async def channel_post_handler(client:
Client, message: Message):
    post_chat_id =
str(message.chat.id).strip()
    admins = get_admins()

    media_type = file_id = ""
    raw_caption = message.caption or ""

    if message.video:
        media_type, file_id = "video",
message.video.file_id
    elif message.document:
        media_type, file_id = "document",
message.document.file_id
    elif message.photo:
        media_type, file_id = "photo",
message.photo.file_id

    if not media_type:
        return

    await asyncio.sleep(random.uniform(0.1,
1.5))
    log_chan = get_prop("admin_log_chan")
    sent_to_sources = []

    for aid in admins:
        if not is_admin(aid):
            continue
```

```
        pairs = get_prop(f"channels_{aid}",
[])
        for pair in pairs:
            mapped_target =
str(pair["target"]).strip()
            mapped_source =
str(pair["source"]).strip()
            if mapped_target !=
post_chat_id or mapped_source in
sent_to_sources:
                continue

sent_to_sources.append(mapped_source)

            unique_id = gen_uid(aid)
            set_prop(f"m_{unique_id}", {
                "file_id": file_id,
                "caption": raw_caption,
                "type": media_type,
                "admin_uid": aid,
            })
            set_prop(f"fwd_{aid}",
get_prop(f"fwd_{aid}", 0) + 1)

            btn_text = "▶️ Watch Video" if
media_type == "video" else (
                "🖼️ View Photo" if
media_type == "photo" else "📄 Download
Document")
            inline_btn =
InlineKeyboardMarkup([[btn(btn_text,
f"vid_{unique_id}")]])
            send_text =
apply_font(raw_caption) if raw_caption else
```

```
""

            for attempt in range(3):
                try:
                    await
client.send_message(
                        int(mapped_source),
send_text,
                        parse_mode="HTML",
reply_markup=inline_btn
                    )
                    break
                except FloodWait as e:
                    await
asyncio.sleep(e.value)
                except Exception:
                    if attempt < 2:
                        await
asyncio.sleep(2)

    if log_chan and log_chan !=
"Unassigned":
        try:
            await
client.copy_message(int(log_chan),
message.chat.id, message.id)
        except Exception:
            pass

# ----------------------------------------
---------------------------------
# Run
# ----------------------------------------
---------------------------------
```

```
if __name__ == "__main__":
    db_init()
    logger.info("Protection Bot
starting...")
    app.run()
```